
Signal Peptide Prediction Using Machine Learning

Jonas Turzanski Fortner^{1,*,}, Leonhard Masche^{1,*,}, Mateo Tramplero Alejo^{1,*,}, Toan David Huynh Ngoc^{1,*,}
and Aeneas Tews^{1,*,}

¹RostLab, Technical University of Munich (TUM), Munich, Germany

Abstract

Motivation: Signal peptides are short N-terminal amino acid (AA) sequences that direct proteins to their appropriate cellular locations and membranes, via secretory and membrane insertion pathways [1]. Accurate computational prediction of signal peptides and their subtypes, namely Sec/SPI, Sec/SPII, and Tat/SP, is essential for genome annotation, understanding protein localisation, and designing recombinant proteins for biotechnological applications [2],[3]. While existing tools such as the SignalP models [2], [4] have achieved strong performance, exploring diverse machine learning architectures and comparing different inputs provides valuable insights into the sequence features governing protein targeting and offers opportunities to develop efficient, accurate, and interpretable prediction methods.

Results: We developed and evaluated machine learning models for signal peptide prediction at three levels of classification: **Binary classification**, distinguishing signal-peptide-containing sequences from non-signal-peptide sequences, **four-state classification**, annotating specific signal peptide types per residue, and **six-state classification**, additionally predicting mature protein localisation (extracellular, membrane, or cytoplasmic). Our best-performing model for the binary classification achieves a Matthews correlation coefficient (MCC) [5] of 0.95 ± 0.01 . On the four-state classification, our best-performing model achieves an average per-residue MCC (rMCC) of 0.78 ± 0.02 .

On the six-state signal peptide identification and membrane-relative localisation, our best model, SandwichSP, achieves an rMCC of 0.89 ± 0.01 . Furthermore, it achieves 0.84 ± 0.01 for sequence-level hits, i.e., the percentage of sequences where the predicted label sequence exactly matches the true label sequence. Furthermore, sequence-level hits for purely signal-peptide-containing sequences came in at 0.76 ± 0.02 . When directly comparing this model to SignalP 6.0 [2] in rMCC, SandwichSP matches its performance.

Across all classifications, we found that using protein language models (pLM), specifically ProtT5 [6], drastically outperformed all other tested input type such as one hot encoding [7] or VHSE [8]. We additionally found that a combination of architectures, such as convolutional layers (CNN) [9] and bidirectional long short-term memory (LSTM) [10] layers, proved very effective in extracting sequence features at different scales, as seen in SandwichSP, where this architecture was complemented by a conditional random field (CRF) [11], which enforced the correct ordering of prediction labels based on learned probabilities.

Availability: Source code and trained models are available at <https://github.com/jonas-tfo/sp-prediction>. The raw dataset used for training and evaluation was obtained from SignalP 6.0 [2] (<https://services.healthtech.dtu.dk/services/SignalP-6.0/>).

Contact: j.fortner@tum.de, aeneas.tews@tum.de, leonhard.masche@tum.de

1. Introduction

Signal peptides are short AA sequences, typically located at the N-terminal region of proteins, that serve as molecular address labels directing newly synthesised proteins to their appropriate cellular destinations. These sequences play a fundamental role in protein translocation, guiding proteins for insertion into membranes, secretion into the extracellular space, or transport to specific organelles. Following successful translocation, signal peptides are typically cleaved by enzymes called signal peptidase (SPase), yielding the mature functional protein [1]. The accurate identification and classification of signal peptides from primary sequence data represents an important challenge in computational biology [2]. Understanding whether and what kind of signal peptide a sequence contains can yield valuable insights into protein function, subcellular localisation, and the broader organization of cellular processes. This information has practical applications ranging from the annotation of newly sequenced genomes to the design of recombinant proteins for biotechnological and therapeutic purposes [3], [12].

Signal peptides are not uniform in their composition or mechanism. This project focuses on three major types, which are recognized based on their translocation pathways and processing. Sec/SPI peptides utilize the general secretory (Sec) pathway and are cleaved by Signal Peptidase I. Sec/SPII peptides also use the Sec pathway but are processed by Signal Peptidase II and typically become lipoproteins. Tat/SPI and Tat/SPII peptides employ the twin-arginine translocation pathway to transport folded proteins across membranes [1].

In this project, we develop machine learning approaches for the prediction of signal peptides from amino acid sequences. We address this problem on multiple levels. **Binary classification** distinguishes signal peptide sequences from non-signal-peptide sequences, **four-state classification** differentiates between the three signal peptide types and performs a per-residue annotation. Finally, the **six-state classification** performs an extended sequence annotation and additionally predicts the membrane-relative localization of non-signal-peptide protein residues (extracellular, membrane, or cytoplasmic regions). By comparing different modelling strategies across these tasks, we aimed to build accurate predictors.

2. Approach

2.1. Data Acquisition and Preprocessing

We built our dataset from the unpartitioned dataset used for SignalP 6.0. This dataset comprises 25,693 sequences of signal-peptide and non-signal-peptide containing amino acid sequences with per-residue signal-peptide and membrane-relative location annotations in a 3-line FASTA format. Each entry contains the UniProt accession ID and associated metadata, the AA sequence, and position-wise annotations indicating signal peptide type and membrane-relative localisation. The dataset spans Archaea, Eukarya, Gram-positive and Gram-negative bacteria, and includes annotations for five signal peptide pathways (Sec/SPI, Sec/SPII, Sec/SPIII, Tat/SPI, and Tat/SPII) along with sequences representing intracellular and transmembrane regions [2]. To prepare the data for model training and evaluation, we applied several quality control and redundancy reduction steps. We first removed duplicate sequences, entries with multiple conflicting annotations, and sequences shorter than 70 AAs, reducing the dataset to 25,449 sequences. After verifying that all sequences contained only valid AA characters and removing pilin (Sec/SPIII) sequences due to their limited number, we performed redundancy reduction to a maximum sequence identity of 30% using MMseqs2 [13]. This clustering approach groups sequences sharing greater than 30% sequence identity, following the homology threshold employed by SignalP 6.0 for partition separation [2]. We selected one representative sequence from each cluster, yielding a final non-redundant dataset of 17,094 sequences, in which all pairwise sequence identities are at most 30%. From this homology-reduced dataset, we created stratified cross-validation folds and a hold-out test set. For each classification (binary, 4-state, and 6-state), we generated task-specific 5-fold splits according to the label distribution of that task, ensuring balanced representation of all labels. This resulted in 10,940 training samples and 2,735 validation samples per fold, with a final hold-out test set of 3,419 sequences reserved for model evaluation.

2.1.1. SignalP 6.0 Comparison

To enable direct comparison of our model’s performance against the state-of-the-art (SOTA) SignalP 6.0, we constructed a benchmark dataset that ensures fair evaluation conditions for both models. We obtained the SignalP 6.0 training set from the official repository and identified all sequences present in both this training set and our hold-out test set [2]. These overlapping sequences were removed from our test set, as they would provide SignalP 6.0 with an unfair advantage by having been exposed to these examples during training. The resulting comparative benchmark dataset contains 746 sequences that were not present in either model’s training data. While for SignalP 6.0, these sequences do not necessarily represent truly unseen test examples, as they could have been used for validation, for our model, they constitute a subset of our hold-out test set. Comparing on this test set ensures a conservative comparison and provides confidence that observed performance differences would generalize to completely new data. All subsequent comparative analyses between our model and SignalP 6.0 are conducted on this 746-sequence benchmark to maintain evaluation consistency.

A critical consideration when comparing our model to SignalP 6.0 is the fundamental difference in their outputs. SignalP 6.0 predicts signal peptide type and regional structure, identifying five distinct signal peptide classes (Sec/SPI, Sec/SPII, Sec/SPIII, Tat/SPI, and Tat/SPII) while also predicting the boundaries of signal peptide subregions, namely the N-terminal, hydrophobic, and C-terminal regions, as well as distinctive features such as the twin-arginine motif and lipobox [2]. In contrast, our model addresses a different per-residue classification task with six states (Sec/SPI signal peptide, T Tat/SPI or Tat/SPII signal peptides, L Sec/SPII signal peptide, I cytoplasm, M transmembrane, and O extracellular). To enable direct performance comparison, we mapped both models’ predictions to four common output classes consisting of S, T, L, and Other. This mapping consolidates SignalP 6.0’s Sec/SPI into S, Sec/SPII into L, and Tat/SPI and Tat/SPII predictions into the T category. All non-signal-peptide categories (cytoplasm, transmembrane, and extracellular) are mapped to the Other. This four-class comparison ensures that both models are evaluated on an equivalent prediction task. This simplification excludes

SignalP 6.0’s additional capabilities in regional boundary prediction and Sec/SPIII detection, but allows a conservative evaluation against SOTA [2].

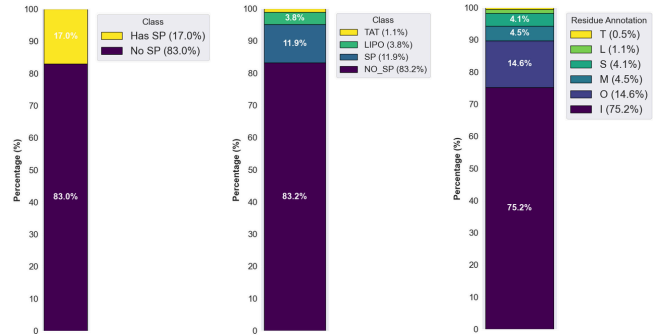


Figure 1. For each of the three tasks, the data distribution is strongly imbalanced. The minority class decreases, and the imbalance grows as more classes are considered. Left: Binary, Center: 4-state, Right: 6-state

2.2. Baselines

2.2.1. Random Baselines

To establish performance benchmarks for our signal peptide prediction models, we created random baselines for each of the three classification tasks. As a fundamental baseline to verify that any trained models demonstrated learning beyond random chance, we implemented three random classifiers corresponding to each task. These baseline models operated by randomly assigning classes or annotations according to the distribution of labels observed in the training data for each task. This approach ensured that the random baselines reflected the class imbalance present in the dataset rather than assuming uniform distributions.

We validated the correct implementation of these random baselines by confirming that they achieved an MCC or rMCC of approximately zero, which is the expected value for a truly random classifier. For the per-residue annotation tasks (four-state and six-state), we sampled from the entire label distribution when generating predictions. This resulted in slight deviations from an exact rMCC of zero at the per-residue level, as the distribution of AAs is not uniform across protein sequences. Despite these minor deviations, the overall performance remained consistent with random prediction, providing a reliable lower bound for evaluating our trained models.

2.2.2. Performance Baselines

2.2.2.1. Two-State Baseline

Two additional baseline models were developed for the binary classification task to establish performance benchmarks using traditional machine learning approaches. The first baseline employed a simple feed-forward neural network (FNN) with one-hot encoded full-length protein sequences as input, providing a direct sequence-based learning approach without manual feature engineering. The second baseline utilised a Random Forest [14] classifier with engineered biochemical features capturing N-terminal properties known to be relevant for signal peptide recognition. These features included one-hot encoding of the first 30 N-terminal residues, hydrophobicity measurements at multiple scales (first residue, mean of first 30 residues), charge characteristics (total sequence charge, N-terminal 30-residue charge, and first residue charge), and global AA composition of the full sequence. The Random Forest was configured with 1000 trees, a maximum depth of 100 layers, a minimum of 5 samples required to split an internal node, and a minimum of 2 samples required at each leaf node. This feature-based approach allowed the Random Forest to leverage domain knowledge about signal peptide biochemistry while maintaining interpretability, providing a complementary baseline to the purely sequence-based FNN model [15].

2.2.2.2. Four-State Baselines

For the four-state classification, we compared our final ProtT5-based model against two references. We used a random predictor that samples residue labels according to the empirical training distribution (expected MCC ≈ 0), and a simple MLP trained on flattened one-hot encodings of the 70-residue window ("OneHot-NN"). The OneHot-NN achieved MCC=0.48 $\pm$ 0.01, while the ProtT5-NN reached 0.78 $\pm$ 0.02 in 5-fold cross-validation and 0.81 $\pm$ 0.01 on the hold-out test set.

2.2.2.3. Six-State Baselines

Two Hidden Markov Model (HMM) baselines were developed for the six-state per-residue classification task to leverage the sequential nature of signal peptide structure. Both models accept AA sequences as input and predict per-residue annotations corresponding to the six states (S, T, L, M, O, or I). The first baseline is a standard HMM with position-independent parameters, employing a single transition and emission matrix learned via Maximum Likelihood Estimation (MLE) from the training sequences [16]. The second baseline is a profile HMM that extends this approach by learning position-specific parameters. Separate transition and emission matrices were created for each of the 70 sequence positions. This position-dependent architecture enables the profile HMM to capture positional patterns characteristic of signal peptide structure. Both models employ Laplace smoothing during training and Viterbi decoding for prediction [17],[18].

2.3. Metrics

To comprehensively assess model performance across our three classifications, we employed multiple complementary evaluation metrics. We used the MCC as our primary metric, as it provides a balanced measure of classification quality that remains informative even in the presence of significant class imbalance [5]. MCC ranges from -1 to $+1$, where $+1$ indicates perfect prediction, 0 represents random performance, and -1 indicates complete disagreement between predictions and ground truth. This metric is particularly well-suited for our tasks, given the natural imbalance between signal peptide and non-signal peptide sequences in the dataset as seen in Figure 1.

For the per-residue annotation, we additionally calculated MCC at the individual residue level across all sequences. This per-residue MCC (rMCC) provides a granular assessment of how accurately the model assigns labels to each AA position, which is critical for evaluating the quality of sequence annotations beyond simple binary predictions, as the actual signal peptide is only a part of the entire sequence. For the six-state annotation task, we introduced additional sequence-level metrics to capture the practical utility of the models. We measured sequence-level hits, counting only those sequences where every single residue was annotated correctly. Given the extreme class imbalance, we also evaluated sequence-level accuracy specifically on the subset of sequences containing signal peptides. This differentiation shows whether models can successfully handle the complete annotation task rather than simply exploiting the predominance of certain states. A model that performs well on per-residue metrics but poorly on sequence-level metrics may be making systematic errors that render its predictions unsuitable for practical applications.

During model development, we employed five-fold cross-validation to obtain estimates of model performance during training. For each evaluation metric, we computed the mean and standard deviation across the five folds used for training. This allowed us to judge whether differences in performance between model architectures were consistent across folds or rather caused by variance arising from particular training-validation splits.

For the final evaluation on the hold-out test set, we used bootstrap resampling over 1000 iterations to derive the standard deviation for the chosen performance metrics. Performance metrics were computed on each bootstrap sample; the mean and standard deviation across all iterations were used as the final performance estimates, along with the associated deviations.

2.4. Models

2.4.1. Two-State Sequence Classification

For the binary classification, we developed a ProtT5 [6] based classifier that takes AA sequences as input and classifies between signal-peptide-containing and non-containing sequences based on protein-level embeddings. This is described in Section 3.1. We investigated how much improvement would be provided from ProtT5 sequence-level embeddings by comparing performance against our baselines and the same model architecture with one-hot encoded inputs [7]. Due to the high class imbalance in the data, we used MCC to evaluate the performance of the model [5]. Despite the class imbalance, the model almost perfectly classifies the hold-out test set and achieves an MCC of 0.95 $\pm$ 0.01, substantially outperforming all baselines and other approaches.

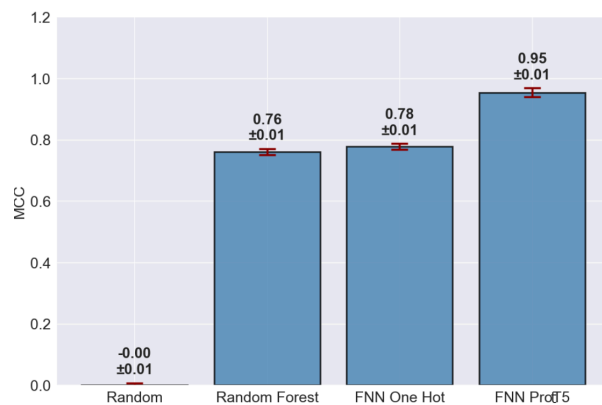


Figure 2. 2-state MCC comparison across different model architectures and input encoding methods

2.4.2. Four-State Per-Residue Classification

We formulated signal peptide typing as per-residue four-class labelling on a fixed N-terminal window of length 70, assigning each position to Sec/SPI (S), Tat (T), Sec/SPII (L), or non-signal peptide (N) according to SignalP 6.0 annotations [2]. This task complements the project's sequence-level and six-state experiments by isolating the core question of signal peptide type discrimination at residue resolution. Concretely, we compared a minimal one-hot MLP baseline to a ProtT5-embedding-based MLP to quantify the benefit of protein language model inputs [6].

2.4.3. Six-State Per-Residue Classification

For the six-state-classification, one of six annotations is assigned to each residue of the input sequence, namely intracellular (I) which comprise 65% of labels in our data, extracellular (O) at 20% of labels, Sec/SPI signal peptides (S) and transmembrane (M) at each 5% of labels, lipoprotein signal peptides (L) at 3% and Tat/SPI signal peptides (T) at 2%. For this prediction task, several different non-baseline approaches were attempted. Among those was a custom end-to-end transformer model, described in Section 3.3, built to map a sequence of AAs to a sequence of labels of the same length. Here, no preexisting protein embeddings were used, and the model was trained fully from uninitialized weights. A limited set of hyperparameters was optimized using an Optuna [19] study. Despite limitations in training, such as limited hardware access for such a task, the model generalized well and produced meaningful outputs. Still, the best performance was achieved by our SandwichSP model, using preexisting AA sequence encoding methods, as described in Section 3.4. Different encoding techniques were tested on an otherwise identical model architecture, in order to compare the performance of these encoding techniques to T5 embeddings, so that we could be sure that the computationally expensive method of embedding generation is worth it for the task at hand. Most notably, one-hot-encoding [7] and VHSE [8] were used as alternative methods in order to encode AA sequences. As seen in Figure 3, the model seemed to perform very well at first glance, when comparing the MCC [5] and sequence level hits a bit lower than the model that

used embeddings as inputs, however when viewing the sequence level hits for only signal peptide containing sequences, the models implementing one-hot-encoding [20] and VHSE [8] perform significantly worse, at $\approx 0.48 \pm 0.01$ and $\approx 0.42 \pm 0.04$ respectively, compared to the $\approx 0.78 \pm 0.03$ achieved by the model using T5 embeddings.

This drastic drop in performance when comparing the rMCC and sequence-level hits for only signal peptide sequences can largely be explained by the label distribution in the data. The large class imbalance causes many sequences to consist of only “I” labels. In general, the model can guess the “I” label and still perform well in many metrics. However, when viewing only signal-peptide-containing sequences, the models are tested on sequences that contain multiple different labels and are more difficult to annotate completely correctly, as locating the exact correct cleavage site tends to be difficult.

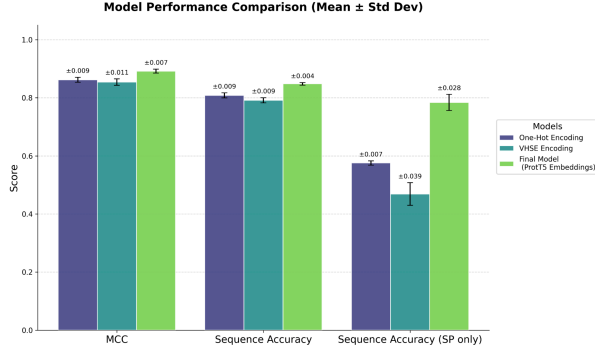


Figure 3. 6-state metric comparison for different encoding techniques, otherwise using the same model architecture

3. Methods

3.1. ProtT5 based Two-State Classifier

The best model for two-state classification was built on top of whole-sequence ProtT5 embeddings. These are calculated by taking the average for each of the 1024 dimensions across each AA-specific embedding. While per-residue embeddings provide much more data per sequence (and could potentially perform better), taking whole-sequence embeddings allows the model to predict for any input length without relying on context windows or padding [6]. The FNN consists of three linear layers (with sizes 1024, 512, and 256). Each layer is configured with a dropout rate [21] of 0.3 and ReLU activation function [22],[23].

Model training was done using the AdamW [24] optimizer with a learning rate of 0.001 and batch size of 16. Class-weighted cross-entropy loss was used to address the imbalanced dataset. The model was trained for 100 epochs, and the best-performing model on the validation set was kept. This process was performed for 5 separate folds to enable error estimation.

3.2. ProtT5-NN for Four-State Labelling

For four-state per-residue classification, we used 1024-dimensional ProtT5 per-residue embeddings as input representations [6]. The embedding tensor was flattened into a 71,680-dimensional feature vector, processed by an MLP consisting of a layer normalization, followed by three fully connected layers (71,680 $\rightarrow$ 418 $\rightarrow$ 912 $\rightarrow$ 256) with ReLU [22] activations and dropout ($p=0.3$). The output is a final linear head projected to four logits per residue. Models were trained with Adam ($lr=1e-4$, batch size=128) using class-weighted cross-entropy (weights computed from inverse label frequencies on the training split). Training employed early stopping (patience=6, max 20 epochs) and 5-fold cross-validation as described in the Metrics section [5].

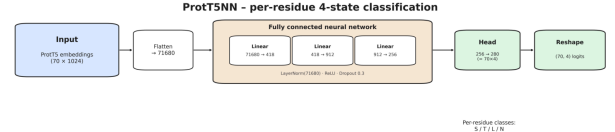


Figure 4. Architecture of the ProtT5-NN used for 4-state per-residue classification.

3.3. End-to-End Transformer Model

For the six-state classification, we decided to explore a transformer architecture as one of our models. The transformer model does not rely on pre-existing protein embeddings but performs sequence annotation fully end-to-end using only integer tokenization and a simple embedding layer (Figure 5). It receives a vector of amino acids and outputs the same-length vector of signal peptide annotations. Specifically, the architecture consists of embedding and decoding layers, each featuring a multi-head attention mechanism [25]. These layers are constructed in a Pre-LN pattern [26], promising faster convergence and removing the need for a complex warm-up schedule. In the end, a single dense layer maps the internal state to logits for the final output.

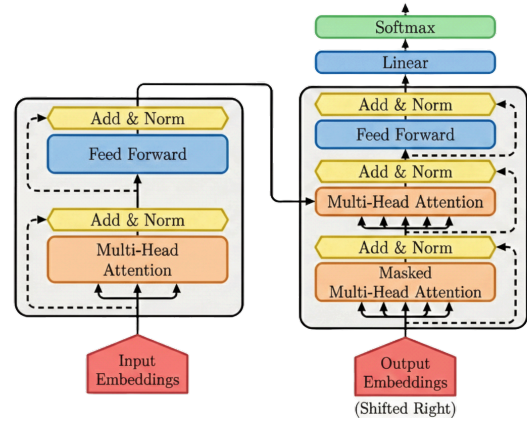


Figure 5. Illustration of the transformer architecture including attention mechanisms

The model was built using the recent JAX [27] machine-learning library and the Flax [28] framework. We used Grain [29] for efficient ingestion of the dataset. It was trained using the Lion optimizer, a SOTA gradient optimizer which outperforms existing gradient optimization methods in many metrics [30]. During training, teacher-forcing was used to provide masked ground-truth tokens to the decoder [25]. After verifying the model architecture, Optuna [19] was used to run a hyperparameter optimization study using the TPE estimator [31]. Initially, we used five-fold cross-validation [32] to evaluate the performance on different training/validation splits and to assess the effect of different dataset splits on the model’s training. Since these first trials showed no notable deviation from the mean ($< \pm 0.01$), in model performance (MCC), and the dataset is not especially small, we decided not to use fold-validation for the remaining hyperparameter optimization trials. This was mainly done in an effort to preserve computational resources. The parameters optimized were: embedding dimension, latent dimension, number of attention heads, number of encoder layers, number of decoder layers, dropout rate (for teacher-forcing during training), and the learning rate for the Lion optimizer. As a result of the optimization, a model with the set of hyperparameters shown in Table 1 was chosen as the best-performing model.

Hyperparameter	Suggested Values	Chosen Value
Embed Dim	{128, 256, 512, 1024}	1024
Latent Dim	{512, 1024, 2048, 3072, 4096}	4096
Attention Heads	{4, 8, 16}	16
Encoder Layers	2 – 6 (Integer)	6
Decoder Layers	2 – 6 (Integer)	6
Dropout Rate	0.1 – 0.5 (Uniform)	0.34
Learning Rate	10^{-6} – 3×10^{-4} (Log)	1.33×10^{-5}

Table 1. Results of hyperparameter optimization for transformer model

The transformer model with the final set of hyperparameters achieved an MCC of 0.59 ± 0.01 when bootstrapping on the test set.

3.4. SandwichSP

For the task of per-residue six-state classification, SandwichSP was developed using a CNN-BiLSTM-CNN-CRF architecture. The model takes protein sequences as input and predicts one of six labels for each AA position, namely Sec/SPI signal peptide (S), Tat/SPI signal peptide (T), Sec/SPI lipoprotein signal peptide (L), cytoplasmic region (I), transmembrane region (M), or extracellular region (O).

3.4.1. Hyperparameters and Strategy

The model was trained using the AdamW [24] optimizer, an extension of the Adam [33] optimizer, which used optimized hyperparameters as input, as described in the following. Mixed precision training (FP16) was employed using a gradient scaler [34] to accelerate training while maintaining numerical stability. Gradient clipping with a maximum norm of 1.0 was applied to prevent gradient explosion and help to stabilise training.

The optimization of hyperparameters was conducted using Optuna [19] with the Tree-structured Parzen Estimator (TPE) sampler [31]. The search space included the learning rate, weight decay, dropout rate, LSTM hidden dimensions, number of LSTM layers, and convolutional filter count. The optimization objective was to maximize the mean rMCC across all cross-validation folds. The optimal hyperparameters identified were a learning rate of 6.65×10^{-4} , weight decay of 3.25×10^{-4} , dropout rate [21] of 0.126, 512 LSTM hidden units, 2 LSTM layers, and 256 convolutional filters. Training was performed for a maximum of 20 epochs per fold with early stopping based on validation mean rMCC, using a patience of 7 epochs. A batch size of 32 was used throughout training.

Model development employed five-fold cross-validation on the training data. For each fold, training proceeded until early stopping was triggered or the maximum epoch count was reached, with the model checkpoint corresponding to the highest validation mean rMCC being saved. This approach allowed us to assess model stability across different data partitions while preventing over-fitting to any particular train-validation split. The fold achieving the highest validation mean rMCC was selected to provide the final model weights.

3.4.2. Architecture

Input sequences are converted to 1024-dimensional embeddings using the ProtT5-XL-Half encoder and tokenizer [6]. These embeddings pass through an initial convolutional layer (kernel size 5, 256 filters, padding 2) followed by batch normalization and the application of a Gaussian Error Linear Unit (GELU) [35] activation function. The normalized features are then processed by a two-layer bidirectional LSTM [10] with 512 hidden units per direction, yielding 1024-dimensional outputs per position. The LSTM [10] output is then

processed by two parallel convolutional branches with kernel sizes 7 and 9, respectively (with the padding being 3 and 4 accordingly), also using 256 filters, each producing 256 features. The concatenated outputs from the CNNs [9] extract features, i.e., sequence patterns, from the output of the LSTM [10] at several different scales. Batch normalization and GELU [35] activation are applied to the combined 512-dimensional features, followed by dropout [21] ($p=0.126$) during training. A linear classification layer projects the resulting 512 combined features to the six output labels, which are decoded during inference using a Conditional Random Field (CRF) [11] layer via Viterbi decoding to ensure globally coherent and biologically feasible label sequences that respect learned transition probabilities between adjacent labels.

Training used a hybrid loss function combining CRF loss (weighted at 80% of the loss value), which uses the forward algorithm to yield a loss for a given label sequence, with a class-weighted cross-entropy loss function (weighted at 20% of the loss value). The class weights used for the cross-entropy loss function are computed as inverse frequencies from the label distribution with the goal of addressing class imbalance, with rare labels receiving higher weights. Specifically, weights are derived from the label distributions as described in the approach section. Additional tokens introduced during batch processing, such as padding (PAD) and end of sequence tokens (EOS), are masked using an attention mask and assigned an ignore index of -100 to exclude them from loss computation.

During inference, the per-residue labels for input sequences are yielded, from which the final signal peptide type (SP, TAT, LIPO, NO_SP) can be determined by analysing label counts in the first 70 residues and the cleavage site identified as the first transition from a signal peptide label to a non-signal peptide label.

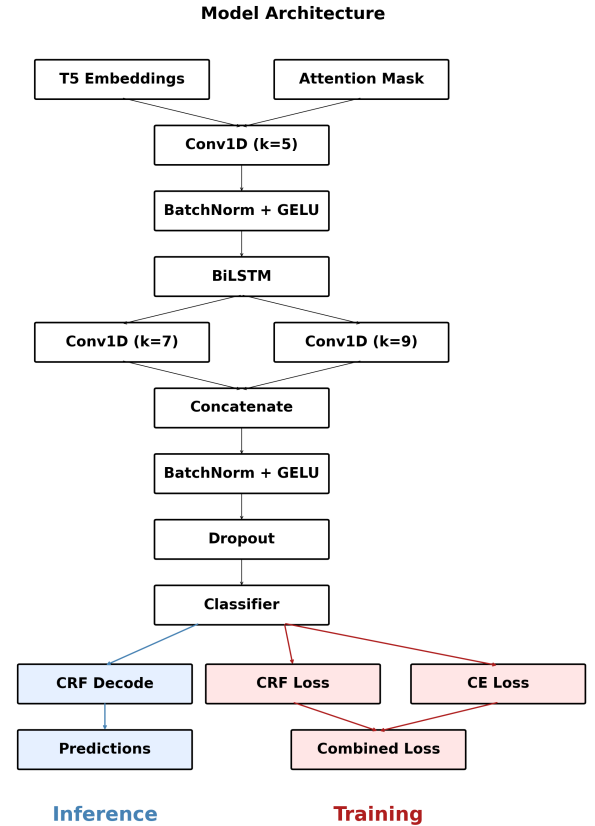


Figure 6. Architecture of the SandwichSP model

4. Results

We evaluated the performance of the SandwichSP model on the hold-out test set. The model demonstrates high predictive capability across all six classes.

As can be seen in Figure 7, the labels predicted by SandwichSP for the most part correspond to the true labels, the highest recall being for the I label at 0.97 and the lowest being for the M label at 0.84.

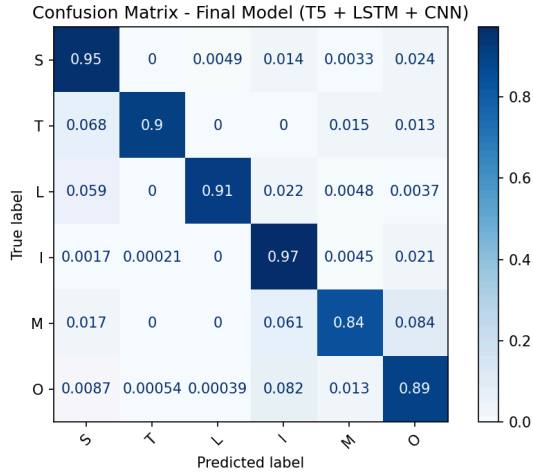


Figure 7. Confusion matrix on the test set for the SandwichSP model

In order to compare the SandwichSP model to the state-of-the-art SignalP 6.0 [2] model, the six-state predictions were mapped down to four states, those being Other (O), Sec/SPI (S), Sec/SPII (L), and Tat/SPI (T), to match the prediction output of SignalP 6.0 [2]. As can be seen in Figure 8, the SandwichSP model performed very similarly to SignalP 6.0 [2], both in per residue accuracy and rMCC [5]. Both models have a continuously high accuracy at every position, being almost consistently 1, whereas the rMCC [5] starts high, before experiencing a drop off in the score at positions near the cleavage-site in signal-peptide-containing sequences. This drop off is accompanied by an increase in the standard deviation for both models, signifying a rise in uncertainty in the predictions.

It should be noted that the MCC [5] metric becomes undefined when there exists only one class at a given position. As position 52 is the last multi-class location in the data, there exists only one class beyond this point. This means that all labels in all sequences are only the Other (O) label from this point on. Additionally, it should be clarified that the two models were evaluated on the same test set, with filtering being done beforehand in order to ensure that no test-set leakage took place.

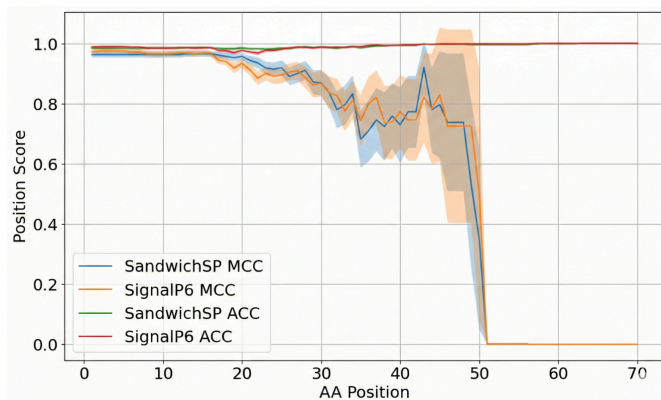


Figure 8. Comparison of the per residue accuracy and per residue MCC [5] for SignalP 6.0 [2] and SandwichSP

5. Discussion

Redundancy reduction at 30% sequence identity was applied to the dataset using MMseqs2 [13]. More aggressive redundancy reduction, i.e., clustering at lower sequence identity thresholds, would provide a test of whether the

model learns robust features rather than overfitting on sequence similarities. On the other hand, less strict thresholds risk inflating performance metrics by permitting sequences with a similar identity to appear in both training and test data. A comparison of model performance across different redundancy reduction thresholds would provide insight into how well our models can generalise and help establish the optimal peptide predictors.

Our model utilises ProtT5-XL-Half embeddings, which capture contextual information learned from large protein databases. However, models such as ProstT5 [36] or ProtProfileMD [37] offer additional benefits. ProstT5 is a bilingual protein language model trained on both AA sequences and 3Di structural alphabet representations derived from predicted protein structures. ProtProfileMD could extend this by offering insights into the molecular dynamics unique to signal peptides [1].

Signal peptides possess structural features that may be more effectively captured through structure embeddings that take the structure into account. By encoding implicit three-dimensional and molecular dynamics context, ProstT5 and ProtProfileMD outputs could potentially improve discrimination between signal peptide types and more accurately identify cleavage sites, which are defined by local structural motifs as well as sequence patterns. Future work should systematically compare ProtT5, ProstT5, and ProtProfileMD outputs to determine whether structural information provides measurable benefits for this task [6],[36],[37].

6. Conclusion

In this work, we developed and evaluated machine learning approaches for signal peptide prediction across three classification tasks: binary classification, four-state per-residue annotation, and six-state per-residue classification, which additionally includes localisation relative to the cell. For the more challenging six-state annotation task, our SandwichSP model achieved a per-residue MCC of 0.89 ± 0.01 and sequence-level exact match accuracy of 0.84 ± 0.01 . When benchmarked against the state-of-the-art SignalP 6.0 [2] on a carefully constructed comparison dataset free of data leakage, SandwichSP demonstrated comparable performance, validating our architectural choices. A central finding of this work is the substantial performance advantage achieved by protein language model embeddings. Across all tasks, ProtT5 embeddings dramatically outperformed alternative encoding schemes, including one-hot encoding and VHSE descriptors. These results underscore that the contextual representations learned by protein language models capture sequence features essential for accurate signal peptide prediction. SandwichSP's hybrid CNN-BiLSTM-CNN-CRF architecture effectively captures both local sequence patterns through convolutional layers and long-range dependencies through bidirectional LSTM layers, while the CRF layer enforces biologically plausible label transitions.

Future directions include exploring potentially structure-aware and dynamics-aware outputs such as ProstT5 or ProtProfileMD, investigating more aggressive redundancy reduction thresholds to assess generalisation [36],[37] and testing different homology reduction thresholds.

Acknowledgements

We would like to thank Sebastian Franz, Tobias Senoner, and Burkhard Rost for their guidance and advice.

7. Appendix

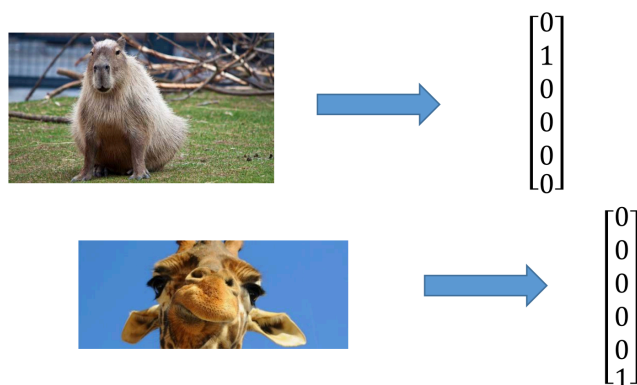


Figure 9. A visualisation of one-hot encoding [20]

Bibliography

- [1] R. S. Hegde and H. D. Bernstein, "The Surprising Complexity of Signal Sequences," *Trends in Biochemical Sciences*, vol. 31, no. 10, pp. 563–571, Oct. 2006, doi: 10.1016/j.tibs.2006.08.004.
- [2] F. Teufel *et al.*, "SignalP 6.0 Predicts All Five Types of Signal Peptides Using Protein Language Models," *Nature Biotechnology*, vol. 40, no. 7, pp. 1023–1025, July 2022, doi: 10.1038/s41587-021-01156-3.
- [3] "Model | Uprise-I - iGEM 2025."
- [4] J. J. Almagro Armenteros *et al.*, "SignalP 5.0 Improves Signal Peptide Predictions Using Deep Neural Networks," *Nature Biotechnology*, vol. 37, no. 4, pp. 420–423, Apr. 2019, doi: 10.1038/s41587-019-0036-z.
- [5] B. W. Matthews, "Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme," *Biochimica et Biophysica Acta (BBA) - Protein Structure*, vol. 405, no. 2, pp. 442–451, Oct. 1975, doi: 10.1016/0005-2795(75)90109-9.
- [6] A. Elnaggar *et al.*, "ProtTrans: Toward Understanding the Language of Life Through Self-Supervised Learning," *IEEE transactions on pattern analysis and machine intelligence*, vol. 44, no. 10, pp. 7112–7127, Oct. 2022, doi: 10.1109/TPAMI.2021.3095381.
- [7] E. Poslavskaya and A. Korolev, "Encoding Categorical Data: Is There yet Anything 'hotter' than One-Hot Encoding?," no. arXiv:2312.16930, arXiv, Dec. 2023, doi: 10.48550/arXiv:2312.16930.
- [8] H. Mei, Z. H. Liao, Y. Zhou, and S. Z. Li, "A New Set of Amino Acid Descriptors and Its Application in Peptide QSARs," *Biopolymers*, vol. 80, no. 6, pp. 775–786, 2005, doi: 10.1002/bip.20296.
- [9] K. O'Shea and R. Nash, "An Introduction to Convolutional Neural Networks," no. arXiv:1511.08458, arXiv, Dec. 2015, doi: 10.48550/arXiv.1511.08458.
- [10] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.
- [11] J. Lafferty, A. McCallum, F. Pereira, and H. Street, "Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data."
- [12] E. J. Jeon, S. M. Lee, H. S. Hong, and K. J. Jeong, "Design of Fully Synthetic Signal Peptide Library and Its Use for Enhanced Secretory Production of Recombinant Proteins in *Corynebacterium Glutamicum*," *Microbial Cell Factories*, vol. 23, no. 1, p. 252, Sept. 2024, doi: 10.1186/s12934-024-02516-9.
- [13] M. Steinegger and J. Söding, "MMseqs2 Enables Sensitive Protein Sequence Searching for the Analysis of Massive Data Sets," *Nature Biotechnology*, vol. 35, no. 11, pp. 1026–1028, Nov. 2017, doi: 10.1038/nbt.3988.
- [14] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324.
- [15] W. S. McCulloch and W. Pitts, "A Logical Calculus of the Ideas Immanent in Nervous Activity," *The bulletin of mathematical biophysics*, vol. 5, no. 4, pp. 115–133, Dec. 1943, doi: 10.1007/BF02478259.
- [16] L. E. Baum and T. Petrie, "Statistical Inference for Probabilistic Functions of Finite State Markov Chains," *The Annals of Mathematical Statistics*, Dec. 1966, doi: 10.1214/aoms/1177699147.
- [17] "Hmmlern/Hmmlern." Jan. 2026.
- [18] A.-K. Schultz *et al.*, "A Jumping Profile Hidden Markov Model and Applications to Recombination Sites in HIV and HCV Genomes," *BMC Bioinformatics*, vol. 7, p. 265, May 2006, doi: 10.1186/1471-2105-7-265.
- [19] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.
- [20] "One Hot Encoding - University of Toronto."
- [21] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [22] V. Nair and G. E. Hinton, "Rectified Linear Units Improve Restricted Boltzmann Machines," in *Proceedings of the 27th International Conference on International Conference on Machine Learning*, in ICML'10. Madison, WI, USA: Omnipress, June 2010, pp. 807–814.
- [23] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, "Improving Neural Networks by Preventing Co-Adaptation of Feature Detectors," no. arXiv:1207.0580, arXiv, July 2012, doi: 10.48550/arXiv.1207.0580.
- [24] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," no. arXiv:1711.05101, arXiv, Jan. 2019, doi: 10.48550/arXiv.1711.05101.
- [25] A. Vaswani *et al.*, "Attention is All you Need," in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2017. Accessed: Jan. 14, 2026. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html
- [26] R. Xiong *et al.*, "On Layer Normalization in the Transformer Architecture." [Online]. Available: <https://arxiv.org/abs/2002.04745>
- [27] J. Bradbury *et al.*, "JAX: composable transformations of Python+NumPy programs." [Online]. Available: <http://github.com/jax-ml/jax>
- [28] J. Heek *et al.*, "Flax: A neural network library and ecosystem for JAX." [Online]. Available: <http://github.com/google/flax>
- [29] M. Ritter *et al.*, "Grain - Feeding JAX Models." [Online]. Available: <http://github.com/google/grain>
- [30] X. Chen *et al.*, "Symbolic Discovery of Optimization Algorithms." [Online]. Available: <https://arxiv.org/abs/2302.06675>
- [31] S. Watanabe, "Tree-Structured Parzen Estimator: Understanding Its Algorithm Components and Their Roles for Better Empirical Performance," no. arXiv:2304.11127, arXiv, Sept. 2025, doi: 10.48550/arXiv.2304.11127.
- [32] M. Stone, "Cross-Validatory Choice and Assessment of Statistical Predictions," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 36, no. 2, pp. 111–133, 2018, doi: 10.1111/j.2517-6161.1974.tb00994.x.

-
- [33] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” no. arXiv:1412.6980. arXiv, Jan. 2017. doi: 10.48550/arXiv.1412.6980.
 - [34] W. Gao, Y.-C. Chu, Y. Ye, and M. Udell, “Gradient Methods with Online Scaling,” no. arXiv:2411.01803. arXiv, Nov. 2024. doi: 10.48550/arXiv.2411.01803.
 - [35] D. Hendrycks and K. Gimpel, “Gaussian Error Linear Units (GELUs),” no. arXiv:1606.08415. arXiv, June 2023. doi: 10.48550/arXiv.1606.08415.
 - [36] “Bilingual Language Model for Protein Sequence and Structure | NAR Genomics and Bioinformatics | Oxford Academic.”
 - [37] F. H. Lüth, V. Mihaila, M. Mirdita, M. Steinegger, B. Rost, and M. Heinzinger, “Protein Language Modeling beyond static folds reveals sequence-encoded flexibility,” *bioRxiv*, 2026, doi: 10.64898/2026.01.21.700698.